

コンピュータアーキテクチャI

担当：佐々木 敬泰

第7回

分岐命令

今日の内容

- 無条件分岐命令
- 条件分岐命令

コンピュータアーキテクチャI

3

無条件分岐命令

- プログラムは基本的に上から順に実行
 - 何度も実行したい場合（ループ）は？
 - 関数や手続き呼び出し（サブルーチン呼び出し）をしたい場合は？

5

MIPSの機械語

- 1命令32bitの固定長
 - MIPSの命令フォーマット
 - Type I
 - 即値（定数）を取る書式
- | | | | |
|----|------|------|--------|
| 命令 | Reg. | Reg. | 即値（定数） |
|----|------|------|--------|
- Type J
 - 分岐（ジャンプ）命令用の特殊フォーマット
- | | |
|----|---------|
| 命令 | 飛び先アドレス |
|----|---------|
- Type R
 - オペランドとしてレジスタのみを取る

命令	Reg.	Reg.	Reg.	shamt	機能
----	------	------	------	-------	----

コンピュータアーキテクチャI

7

シラバス

1. イントロダクション
2. コンピュータの構成
3. 数の表現
4. 様々なプロセッサと命令セット
5. 基本命令
6. メモリアクセス命令
7. 分岐命令
8. 高級言語からアセンブラへの変換
9. プログラムの翻訳と起動
10. 加減算
11. 乗算アルゴリズムと乗算器
12. 除算アルゴリズムと除算器
13. 浮動小数点
14. 浮動小数点の乗算と誤差
15. 定期試験

コンピュータアーキテクチャI

2

無条件分岐

4

ループ文（永久ループ）

- 同じ処理を複数回繰り返す（ループ処理）

```
addi $1, $1, 1    # $1=$1+1
addi $1, $1, 1    # $1=$1+1
addi $1, $1, 1    # $1=$1+1
addi $1, $1, 1    # $1=$1+1
addi $1, $1, 1    # $1=$1+1
:                :
```

- 分岐命令を使って実行する場所を変える

```
L1: add  $1, $1, 1    # $1=$1+1
```

ラベル j L1
ジャンプ命令（ラベルの位置に制御を移す）

6

機械語(Type J)

- オペコード（命令）と即値を32ビットに詰込む

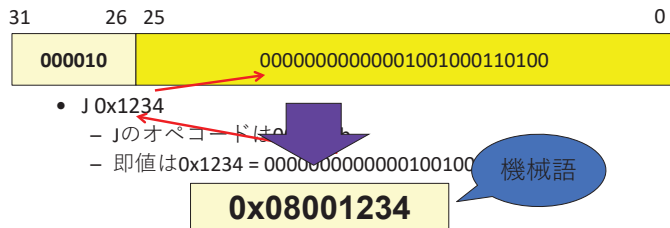


- J 0x1234
 - Jのオペコードは000010b
 - 即値は0x1234 = 00000000000000001001000110100b

8

機械語(Type J)

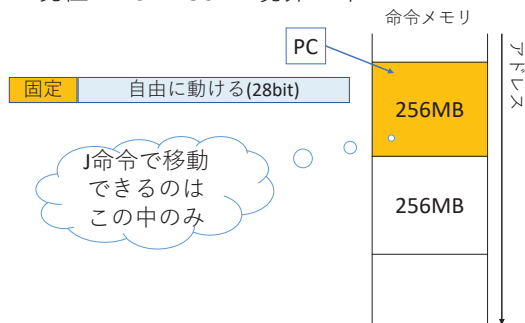
- オペコード（命令）と即値を32ビットに詰込む



9

J命令の飛べる範囲

- 現在のPCの256MB境界の中



- この範囲外に飛ぶ場合は、JR命令を使う

11

JR命令

- J命令は飛び先が即値（固定値）
- 実行時に飛び先が決まる場合は？
- J命令で飛べないところにも飛びたい場合は？
 - JR(Jump Register)命令
 - Type R命令
 - JR \$2 # PC = \$2
 - \$2番地に飛ぶ
 - \$2の値が4の倍数でない場合は例外が発生

13

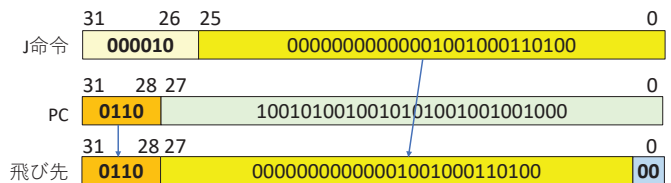
JAL(Jump And Link)命令

- 関数呼び出し
 - 任意のアドレスにジャンプ
 - 戻ってくるためには元の位置の記憶が必要
- JAL命令を使う
 - J命令の動作に加え、「PCの値+4」を\$31にコピーする
 - JALの使い方はJと同じ
 - JAL 0x1230 ← 飛び先アドレス
 - JAL add ← 実際にはラベルを使うことが多い

15

J命令の飛び先

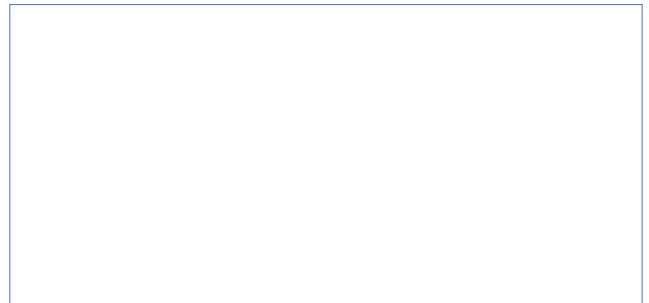
- J命令の飛び先（即値）は26bit
 - そのままでは $2^{26}-1$ 番地までしか飛べない(;ω;)
 - 命令にも整列制約がある（4の倍数になる）
 - ⇒ 下位2bitは必ずゼロなので省略
 - 足りない4bitは？
 - ⇒ PC（プログラムカウンタ）の上位4bitを利用



10

J命令の飛び先

- J命令が40230120₁₆番地に置かれていた時、このジャンプ命令が飛べるのは何番地から何番地までか計算せよ。



コンピュータアーキテクチャI

12

関数呼び出し

- 関数（手続き）呼び出し
- ```

int add(int a, int b){ /* aとbの和を返す関数 */
 return a+b;
}

int main(){ /* ここから開始 */
 int x, y, ans;
 x = 10;
 y = 20;
 ans = add(x, y);
 printf("%d + %d = %d\n", x, y, ans); /* 結果を出力 */
 return 0;
}

```
- 飛んだ場所に  
戻ってくる必  
要がある

14

## 再掲 MIPSのレジスタ割り当て

- レジスタの用途

| 名前            | レジスタ番号  | 用途        | 呼び出し時に退避 |
|---------------|---------|-----------|----------|
| \$zero        | 0       | 常にゼロ      | 不要       |
| ● \$v0 - \$v1 | 2 - 3   | 結果、式の評価   | 不要       |
| ● \$a0 - \$a3 | 4 - 7   | 関数の引数     | 不要       |
| \$t0 - \$t7   | 8 - 15  | 一時的な変数    | 不要       |
| \$s0 - \$s7   | 16 - 23 | 退避        | 必要       |
| \$t8 - \$t9   | 24 - 25 | 予備の一時変数   | 不要       |
| \$gp          | 28      | グローバルポインタ | 必要       |
| \$sp          | 29      | スタックポインタ  | 必要       |
| \$fp          | 30      | フレームポインタ  | 必要       |
| ● \$ra        | 31      | 戻りアドレス    | 必要       |

\$26 - \$27はオペレーティングシステム用に予約されている

16

## コンパイル結果（呼び出し側）

- Cプログラム（抜粋）

```
x = 10;
y = 20;
ans = sum(x, y);
```

- アセンブラ（抜粋）



17

## コンパイル結果（関数定義部分）

- Cプログラム

```
int sum(int a, int b){ /* aとbの和を返す関数 */
 return a+b;
}
```

- アセンブラ



18

## 条件分岐

「もし、合計金額が所持金より多ければエラーを出す」  
ようなプログラム

19

## 条件分岐

- 条件を満たした場合にジャンプする
- 条件とは？
  - 二つのレジスタの値が等しい
  - 二つのレジスタの値が異なっている

20

## beq(branch on equal)命令

- 二つのレジスタ値が同値だったら分岐
  - beq \$1, \$2, 分岐先  
⇒\$1と\$2が同値なら分岐先へ飛ぶ
- 例)

```
if($1 == $2) jump to L1
beq $1, $2, L1
$1と$2が等しければここをスキップ
add $1, $0, $0
L1: add $1, $1, $2
```

21

## bne(branch on not equal)命令

- 二つのレジスタ値が異なっていた分岐
  - bne \$1, \$2, 分岐先  
⇒\$1と\$2が同値でなければ分岐先へ飛ぶ
- 例)

```
if($1 != $2) jump to L1
bne $1, $2, L1
$1と$2が異なった値ならここをスキップ
add $1, $0, $0
L1: add $1, $1, $2
```

22

## if-then-else文

- もし...ならば..., そうでなければ...
- C言語

```
if(a == 10){
 x = x + 20;
}else{
 x = x + 30;
}
```

レジスタ割当て  
a: \$1  
x: \$3

- アセンブラ

```
addi $2, $0, 10
bne $1, $2, L2
addi $3, $3, 20
j L1
L2: addi $3, $3, 30
L1:
```

# if(\$1 != 10) jump to L2  
# x = x + 20  
# Jump to L1  
# x = x + 30

等しくな  
ければ

23

## 条件分岐

- 二つのレジスタの値により分岐するか決定

```
if(a==b){
 /* aとbが等しければ実行 */
}
if(a!=b){
 /* aとbが等しくなければ実行 */
}
```
- 大小関係は？

- if(a > 10) { /\* aが10より大きければ \*/
- if(x >= 20) { /\* aが20以上ならば \*/

➡ slt(set on less than)命令を使う！

24

## 比較演算命令

### • slt命令(set on less than)

slt \$1, \$2, \$3

- もし、\$2 < \$3なら\$1を1に、そうでなければ\$1を0にする命令

【slt命令の疑似コード】

**slt \$1, \$2, \$3**

if(\$2 < \$3){

    \$1 = 1;

}else{

    \$1 = 0;

}

## 比較命令の種類

### • slt

- 符号ありの整数比較

### • sltu

- 符号なし(set on less than unsigned)

### • slti

- 符号ありで即値との比較(set on less than immediate)

### • sltui

- 符号なしで即値との比較(set on less than immediate)

- Set on Less Than命令しかない
- 「以下」や「以上」等が必要ならオペランドの順番や後続の分岐命令で工夫

## C言語のif文の実装例

### • C言語

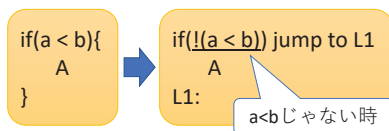
```
int a, b, c;
if(a < b){
 c = 10;
} else {
 c = c + 1;
}
```

### • レジスタ割当て

a:\$4  
b:\$5  
c:\$6

### • アセンブラ

```
slt $2, $4, $5
beq $2, $0, L1
addi $6, $0, 10
j L2
L1: addi $6, $6, 1
L2:
```



## 練習：C言語のif文の実装

### • C言語

```
int a, b, c;
if(a <= b){
 c = 10;
} else {
 c = c + 1;
}
```

### • レジスタ割当て

a:\$4  
b:\$5  
c:\$6

### • アセンブラ

## C言語のwhile文の実装例

### • C言語

```
int func(int a, int b){
 while(a >= b){
 b = b + 1;
 }
 return b;
}
```

### • レジスタ割当て

a:\$4  
b:\$5

### • アセンブラ

```
func:
L1: slt $3, $4, $5
 bne $3, $0, L2
 addi $5, $5, 1
 j L1
L2: add $2, $5, $0
 j $31
```

## 練習：C言語のwhile文の実装

### • C言語

```
int func(int a, int b){
 while(a > b){
 b = b + 1;
 }
 return b;
}
```

### • レジスタ割当て

a:\$4  
b:\$5

### • アセンブラ

## C言語のwhile文の実装（最適化）

### • C言語

```
int func(int a, int b){
 while(a > b){
 b = b + 1;
 }
 return b;
}
```

### • レジスタ割当て

a:\$4  
b:\$5

通常、ループの中は何回も実行されるので、ループ内の命令削減は効果的！

## コンパイラによる最適化

### • C言語

```
int func(int a, int b){
 while(a > b){
 b = b + 1;
 }
 return b;
}
```

### • レジスタ割当て

a:\$4  
b:\$5